

DEEP LEARNING

Muhammad Ahmad 25280065 MS(AI)

https://github.com/Ahmad-Gill/DeepLearning/blob/main/25280065_DeepLearning_Assignment1.ipynb

Question No:1

Part A: Data Preprocessing and Exploratory Analysis

The dataset was explored for structure, missing values, and feature encoding. Visualizations were created for class distribution, feature relationships, and correlations.

See the images and code in the .ipynb file for detailed analysis.

Missing Values Handling Strategy:

1. **Numerical Features (minimum_nights, amenity_score, number_of_reviews, availability_365):**
 - **Strategy:** Impute missing values with the median.
 - **Reason:** The median is robust to outliers and is suitable for numerical features with missing values.
2. **Categorical Features (neighbourhood_group, room_type):**
 - **Strategy:** Impute missing values with the mode (most frequent value).
 - **Reason:** The mode represents the most common category, making it a reasonable choice for imputation.

Encode Categorical Variables:

I used One-Hot Encoding because the categorical variables have no natural order. Label encoding would introduce false ranking, while one-hot encoding treats each category independently, which is more suitable for neural networks.

Normalize Numerical Features:

- Neural networks use gradient descent, which works best when features are centered around 0.
- Normalization handles outliers better than Min-Max scaling.
- It speeds up convergence and improves stability during training.

Correlation Matrix Analysis

From the heatmap, all correlation values are very small (close to 0). This indicates that the numerical features are mostly independent of each other.

- **minimum_nights** has almost no correlation with other variables (values around 0.02 to 0.13).
- **amenity_score** is nearly independent from all features.
- **number_of_reviews** and **availability_365** show the highest correlation at 0.17, but this is still weak.
- Negative correlations are extremely small (e.g., -0.08).

Feature Influence Analysis

- **Most influential features:**
 - **Amenity score:** Increases clearly from budget to luxury classes, showing strong separation in boxplots.
 - **Neighbourhood group:** Higher classes are concentrated in Manhattan, while cheaper classes are mostly in Bronx, Queens, and Brooklyn.
 - **Room type:** Shared and private rooms dominate lower price classes, while higher classes rarely contain them.
- **Moderate influence:**
 - **Minimum nights:** Slightly higher in expensive listings but with noticeable overlap.
- **Weak influence:**
 - **Number of reviews** and **availability_365** show heavy overlap across classes and provide limited predictive value.

Dominance Check:

The correlation matrix shows all correlations below 0.2, indicating no feature is suspiciously dominant and no multicollinearity exists.

Conclusion:

Amenity score, **neighbourhood group**, and **room type** are the main predictors, while the remaining features contribute only minor information.

PART B

Final Sigmoid Training Accuracy: 0.5642723260172925

Final Sigmoid Validation Accuracy: 0.5588875453446192

Final ReLU Training Accuracy: 0.5642723260172925

Final ReLU Validation Accuracy: 0.5588875453446192

The ReLU network achieves higher accuracy and improves more steadily across iterations, while the sigmoid network plateaus early due to vanishing gradients. The accuracy curves show better gradient flow and optimization efficiency for ReLU compared to sigmoid.

Simple Gradient Magnitude Comparison:

- **Sigmoid Activation:**
 - Gradients for both layers (W1 and W2) decrease quickly to very small values.
 - This is due to the **vanishing gradient problem**, where the network struggles to update weights in deeper layers.
- **ReLU Activation:**
 - Gradients remain strong and stable across iterations.
 - This helps avoid the **vanishing gradient problem**, allowing faster and more effective learning.

Conclusion:

- **Sigmoid** has vanishing gradients and slows down learning.
- **ReLU** provides stronger gradients, enabling better learning, especially in deeper layers.

PART C (a) Gradient-Based Feature Attribution (Analytical)

DATE: _____

DAY: _____

Question No: 1 C (a)

input layer = $X = R^{m \times d}$ use matrix instead of vector.

First hidden layer (1)

$A' = XW' + b'$ before activation.

output of First hidden layer

$$X' = g(A')$$

Second hidden layer

$$X^2 = g(A^2)$$

Output layer

$$X^3 = g(A^3)$$

$$\hat{y} = X^3$$

Loss function.

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

Gradient

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial L}{\partial X^3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial X^3} = \frac{\partial L}{\partial X^3} = \hat{y} - y$$

DATE: _____

DAY: _____

$$\frac{\partial L}{\partial A_1} = \frac{\partial L}{\partial X_2} w_2^T \quad \frac{\partial L}{\partial X_1} = \frac{\partial L}{\partial A_1} \cdot g(X_1)$$

$$\frac{\partial L}{\partial X_0} = \frac{\partial L}{\partial A'} w_1^T$$

That gives the gradient of loss with respect to X^0 input.

magnitude of gradients.

$$\left\| \frac{\partial L}{\partial X} \right\| = \sqrt{\sum_{i=1}^n \left(\frac{\partial L}{\partial X_i} \right)^2}$$

Pseudocode

- initialize weights, biases and input matrix.

- forward pass.

compute preactive first hidden layer $X_1 = X \cdot w_1 + b_1$

apply activation $A_1 = g(X_1)$

preactivation for second layer $X_2 = A_1 \cdot w_2 + b_2$

$A_2 = g(X_2)$

$X_3 = A_2 \cdot w_3 + b_3$

$\hat{y} = \text{Softmax}(X_3)$

Compute loss

$L = -\text{Sum}(X \cdot \log(\hat{y}))$

Back propagation.

$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial X}$

DATE: _____

DAY: _____

more - making them more important for the model's prediction.

Thus the gradient magnitude helps identify the most influential ones. Features.

Part C

interpretation of result and Relation with EDA

Feature importance

The ReLU activation leads to larger gradients, so features have a stronger influence on model's output compared to sigmoid, which has smaller gradients.

Alignment with part A

In part A we saw that room-type, neighbourhood, group were important for predicting price-class. These features also show up as the most important in the gradient analysis, confirming their role in the model.

Feature room-type or neighborhood group are ranked highly in both activations reinforcing their importance.

DATE: _____

DAY: _____

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y} \cdot w_i^T$$

$$\frac{\partial L}{\partial A^2} = g'(x_i)$$

$$\frac{\partial L}{\partial x_i} = w_i^T \cdot g'(x_i)$$

$$\frac{\partial L}{\partial A^1} = g'(x_i); \quad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial A^1} \cdot w_i^T$$

Compute gradient magnitude.

$\text{Sort}(\text{Sum}((\frac{\partial L}{\partial x_i})^2 / m))$ for each feature.

output the ranked features based on gradient magnitude.

Justification of gradient of magnitude

It simply explains that how much change a feature will effect the loss function.

- Large gradient magnitude, then small changes in x_i causes a signitade change in loss. meaning the feature is important for the ~~two~~ model's prediction. and vice versa for small magnitude.

Feature with large gradients cause the model's

DATE: _____

DAY: _____

Pseudo-code

initialize weights, biases and input matrix

Forward pass

$$X = AW + b$$

$$\hat{Y} = \text{Sigmoid}(X)$$

calculate loss function.

Backward pass

$\frac{dL}{dX}$ for all X matrix.

calculate gradient magnitude using $\frac{dL}{dX}$.

Rank the features based on magnitude.

return ranked features and magnitudes.

PART C (b) Implementation and Results

Sigmoid Gradient Magnitudes (Layer 1, Layer 2): (np.float64(9.36954925942018e-07), np.float64(0.00010356942551313053))

ReLU Gradient Magnitudes (Layer 1, Layer 2): (np.float64(8.760402273466163e-05), np.float64(9.502625622212074e-05))

Top 5 Features based on Gradient Magnitude (Sigmoid):

Feature 2: Magnitude 5.671747887534842e-06

Feature 17: Magnitude 2.4554686033288764e-06

Feature 21: Magnitude 2.1868746012511756e-06

Feature 29: Magnitude 2.085541721287489e-06

Feature 23: Magnitude 1.4147187647097512e-06

Top 5 Features based on Gradient Magnitude (ReLU):

Feature 21: Magnitude 0.00019624775598844538

Feature 13: Magnitude 0.00018140778850334264

Feature 26: Magnitude 0.00017456149210332933

Feature 15: Magnitude 0.0001678596070733991

Feature 18: Magnitude 0.00014275501947500467

PART D Test Evaluation and Generalization Analysis

Test Accuracy with Sigmoid: 0.5588875453446192

Test Accuracy with ReLU: 0.5588875453446192

Part D

Both sigmoid and ReLU has test accuracy of

0.558

The model shows the similar behaviour on train, validate and test about ~50% accuracy with no major overfitting and underfitting.

Generalization failure

Part A, reon-type are important but may not fully explain the target variable.

Part C, Both activations shows similar behaviour

DATE: _____

DAY: _____

suggesting the model is not capturing complex patterns.

Fails on test Set:-

The model likely underfit because its too simple to learn from the data which is why the test accuracy is low.

Solution

Increase model complexity (more layers) and use regularization to improve generalization.

Question No 2 Bias Gradients, Parameter Sharing, and Activation Effects

Question No: 2

Backpropagation considering shared bias b

1) gradient w.r.t output layer

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial L(\hat{y}, y)}{\partial \hat{y}}$$

2) gradient w.r.t hidden layer 2 weights.

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial U}$$

$$\frac{\partial \hat{y}}{\partial U} = h_1$$

DATE: _____

DAY: _____

3) gradient w.r.t hidden layer 1:

$$\frac{\partial L}{\partial h_i} = U^T \frac{\partial L}{\partial \hat{y}} g'(U h_i + b)$$

we derivat of activation function.

4) gradient w.r.t to Bias

calculate gradient of shared Bias

$$\frac{\partial L}{\partial b} = \sum \left(\frac{\partial L}{\partial h_i} g'(U h_i + b) \right)$$

add contribution from both layer.
hidden and output.

for output layer

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} g'(U h_i + b)$$

Since gradient appears in both layer.

$$\frac{\partial L}{\partial b} = \sum \left(\frac{\partial L}{\partial h_i} g'(U h_i + b) \right) + \frac{\partial L}{\partial \hat{y}} g'(U h_i + b)$$

Optimization and convergence effect.

1) shared bias Model

- The same bias parameter b is shared between both layers. This coupling introduces a

DATE: _____

DAY: _____

shared gradient that affects the ~~out~~ updates to the bias.

- the shared bias can help reduce the degrees of freedom in the model, making it more constraints but easier to optimize since the bias is common across the layers.

Independent bias Model:

• Each layer has its own bias parameters b_1 for the hidden layer and b_2 for the output layer.

- This increases the degrees of freedom, allowing each layer to learn biases independently, which may lead to more complex and potentially more flexible models.

Impact on Convergence:

- Yes, bias sharing can effect convergence speed or stability during gradient-based training.
- In shared-bias-model, means the model has less parameters. which should speed up the convergence since fewer parameters to be optimized.
- Independent bias model has slower convergence, since it has more parameters.

Shared bias help stabilized the model when training.